[Chap 16] Business Plan

Linear programming

Linear Programming (LP)

- One of the representative optimization techniques

- Linear programming models – Both the objective function and the constraint equations are first-degree equations

  – All variables are real values – If the above condition is not met, it is not a linear programming model

  - If a non-linear function is involved, use NonLinear Programming (NLP) •
  If there is a condition that the value must be an integer, use Integer Programming (IP)

- Linear programming models occupy a very important position in mathematical programming models.

  – There are also various mathematically well-researched and efficient algorithms for finding optimal solutions.

  – Although a simple model, it has a wide range of applications and serves as the foundation for other complex optimization models

Linear programming example

한 빵집에서 케이크와 바게트를 판매하고 있다. 케이크 하나를 만드는 데에는 3컵의 밀가루가 필요하고, 오븐에서 45분간 구워야 한다. 바게트는 8컵의 밀가루가 필요하고, 오븐에서 30분간 구워야 한다. 그런데, 지금 빵집에는 20컵의 밀가루가 있고, 영업시작 전까지 3시간만 오븐을 이용할 수 있다. 케이크의 가격은 하나에 $100이고, 바게트는 $60이다. 오늘 빵집의 수익을 최대화하려면 케이크와 바게트를 몇 개씩 만들어야 하겠는가?

• Assuming all produced cakes and baguettes are sold

• Structure of the problem

– 1) Must determine the production quantity of cakes and baguettes ÿ Decision variable

– 2) I want to maximize profit ÿ Objective function

– 3) Flour and oven time are limited ÿ Constraints

ÿ Three basic elements of a linear programming model

Linear Programming Examples (Continued)

• Definition of decision variables

– X1: Cake production

– X2: Baguette production

• Definition of Objective

Function – The objective is to

maximize profit – Profit = Cake Price * Cake Quantity + Baguette Price * Baguette Quantity

```
Maximize 10*X1 + 6*X2
```

• Definition of Constraints

– Limited amount of flour and oven usage time – Product yield

cannot be negative

```
subject to
  3*X1 +  8*X2 <= 20     (밀가루 양 제약조건)
 45*X1 + 30*X2 <= 180    (오븐 가동시간 제약조건)
      X1,X2 >= 0
```

Linear Programming Examples (Continued)

- Summary of the entire linear programming model

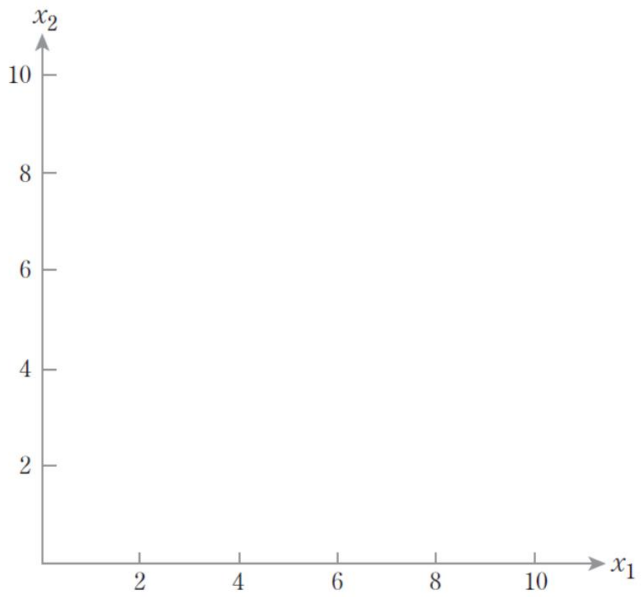
```
Maximize 10*X1 + 6*X2

subject to
           3*X1 +  8*X2 <=  20    (밀가루 양 제약조건)
          45*X1 + 30*X2 <= 180    (오븐 가동시간 제약조건)
                  X1, X2 >= 0

(여기서, X1 = 케이크의 생산량, X2 = 바게트의 생산량)
```

Optimum solution calculation using graphical methods

- Graphical solution method
  – A method of finding the 'feasible region' within the range of constraints by representing constraints as polygons in a space where each decision variable is represented as axes, and finding the optimal solution within this feasible region. • A

space where each decision variable is represented as axes.

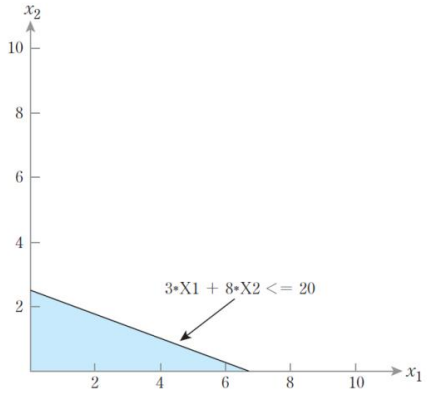
– In the example, since there are two variables, it becomes a 2-dimensional space.



[그림 16–1] 결정변수 값의 좌표공간

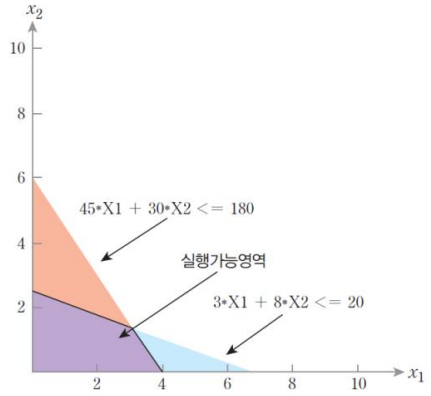
Calculation of Optimum Solution Using Graphical Methods (Continued)

- Constraints on the amount of flour
  - Represent the area of 3*X1+8*X2<=20 in coordinate space (area marked in blue)



[그림 16-2] 밀가루 양 제약조건 충족 영역

- Oven operating time constraints
  - Represent the area of 45*X1+30*X2<=180 (overlaid in red)



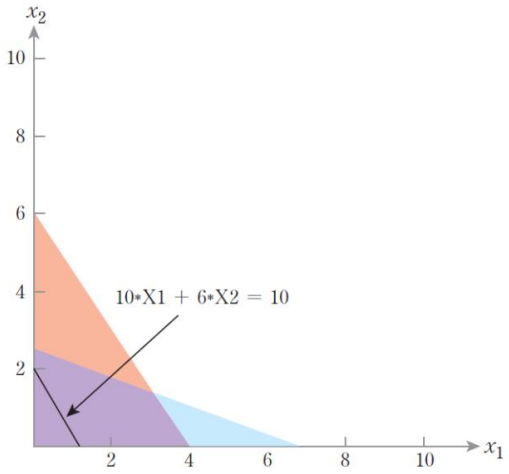
[그림 16-3] 모든 제약조건을 충족하는 실행가능영역

- The overlapping area of the two regions (purple in the figure)
  - It is called a feasible region because it satisfies all constraints – Each point within the feasible region is a feasible solution

Calculation of Optimum Solution Using Graphical Methods (Continued)

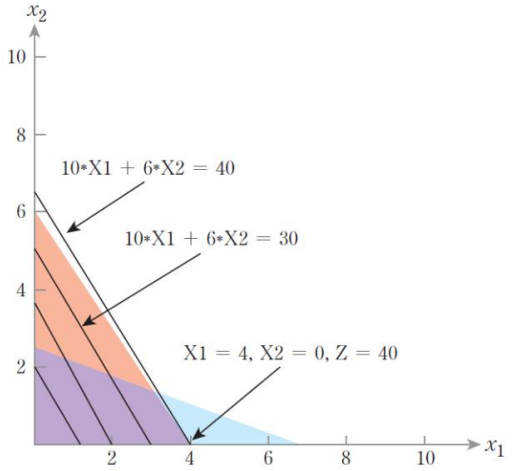
- We just need to find the point that maximizes profit among the executable solutions within the feasibility domain.

- First, all possible points capable of generating a profit of $10 lie on the black line in Figure 16-4 – there are infinitely many possible solutions for generating $10.



[그림 16–4] 수익 $10를 창출하는 실행가능해

- Examine whether there are points capable of generating $20, $30, and $40 – There are infinitely many (X1, X2) combinations capable of generating up to $30 – Among the lines generating $40, the only point within the feasible region is (4, 0).
  - If you achieve a target profit higher than that, it becomes possible within that range. There are no points in



[그림 16–5] 도해법에 의해 구해진 최적해

Therefore, the optimal solution is X1=4, X0=0, and the profit is $40.

Derivation of the optimal solution for a general linear programming model

- Graphical methods are difficult to apply with three or more variables
  - Geometric representation itself is impossible starting from 4 decision variables

- Solution of the general linear programming model
  - Simplex method: Algebraic without relying on geometric graphs

    Techniques for finding the optimal solution of a linear programming model

    using • There are also various other techniques
  - Utilizing various software that calculates the optimal solution

Deriving the optimal solution for a linear programming model using PuLP

- PuLP Library: Supports calculating the optimal solution for linear programming models in Python
  - Installation of PuLP: pip install pulp

- Import so that all function names in pulp can be used directly

```
from pulp import *
```

- Create a linear programming model using the LpProblem command, assign an arbitrary name, and set whether the goal is maximization or

  minimization – set to 'LpMaximize' if the goal is maximization, and 'LpMinimize' if the goal is minimization.

```
LP = LpProblem("bakery_problem", LpMaximize)
```

Deriving the Optimal Solution for a Linear Programming Model with PuLP (Continued)

- Set decision variables using the LpVariable command

```
1  X1 = LpVariable("cake")
2  X2 = LpVariable("baguette")
```

- Setting the Objective

  Function – Add the objective function equation to the variable LP, which was created and saved from the model earlier, as follows

```
LP += 10*X1 + 6*X2
```

- Add constraints – Note that

  inequality signs must be entered as well.

```
1  LP += 3*X1 + 8*X2 <= 20
2  LP += 45*X1 + 30*X2 <= 180
3  LP += X1 >= 0
4  LP += X2 >= 0
```

Deriving the Optimal Solution for a Linear Programming Model with PuLP (Continued)

• .solve() : Solves the problem and derives the optimal solution

```
LP.solve()
```

```
Optimal - objective value 40
After Postsolve, objective 40, infeasibilities - dual 0 (0), primal 0 (0)
Optimal objective 40 - 1 iterations time 0.002, Presolve 0.00
Option for printingOptions changed from normal to all
Total time (CPU seconds):       0.01   (Wallclock seconds):       0.00
```

– The final objective function value is 40, which is identical to the result of the graphical solution.

• Check the values of each variable and the objective function value in the

optimal solution – Name (.name), value (.varValue), and objective function value (.objective) for each element of LP.variables()

output of power

```
1  for v in LP.variables():
2      print(v.name, '=', v.varValue)
3  print("Total profit is ", value(LP.objective))
```

Deriving the Optimal Solution for a Linear Programming Model with PuLP (Continued)

- Full code and output

예제 ex16-1.py

```
1  from pulp import *
2
3  LP = LpProblem("bakery_problem", LpMaximize)
4  X1 = LpVariable("cake")
5  X2 = LpVariable("baguette")
6  LP += 10*X1 + 6*X2
7  LP += 3*X1 + 8*X2 <= 20
8  LP += 45*X1 + 30*X2 <= 180
9  LP += X1 >= 0
10 LP += X2 >= 0
11 LP.solve()
12 for v in LP.variables():
13     print(v.name, '=', v.varValue)
14 print("Total profit is ", value(LP.objective))
```

```
baguette = 0.0
cake = 4.0
Total profit is  40.0
```

Decision-making example using linear programming

'행복한 소시지'는 소시지를 만드는 기업이다. 이 공장은 닭고기, 소고기, 돼지고기, 전분을 배합해 소시지를 만든다. 재료들은 하루에 닭고기 200파운드, 소고기 300파운드, 돼지고기 150파운드, 전분 400파운드가 들어오고, 각 원료의 가격은 파운드 당 $0.2, $0.3, $0.5, $0.05이다.

'행복한 소시지'는 원료의 배합에 따라 일반 소시지, 소고기 소시지, 고기만 소시지를 판매한다. 일반 소시지는 소고기와 돼지고기를 합친 비율이 10% 이하이고, 닭고기의 비율이 20% 이상이어야 한다. 소고기 소시지는 소고기의 비율이 75% 이상이어야 한다. 고기만 소시지는 전분이 들어가지 않고, 소고기와 돼지고기를 합친 비율이 50% 이하여야 한다.

일반 소시지는 파운드 당 $0.9, 소고기 소시지는 $1.25, 고기만 소시지는 $1.75에 판매된다. 이때 '행복한 소시지'의 수익이 최대화되는 생산량을 구하시오.

Decision-making examples using linear programming (continued)

• Decision variable: The quantity of each raw material input into the production of each sausage

– Expressed in the form of Xij (i = ingredient, j = sausage)

```
Xij = 각 원료가 소시지의 생산에 투입되는 수량
i = c, b, p, s (c: chicken, b: beef, p: pork, s: starch)
j = r, b, m (r: regular, b: beef, m: meat)
```

• Objective function: Maximizing profit

– Profit per unit is the price of each sausage minus the price of the raw materials

```
Maximize 0.7*Xcr + 0.6*Xbr + 0.4*Xpr + 0.85*Xar
       + 1.05*Xcb + 0.95*Xbb + 0.75*Xpb + 1.20*Xab
       + 1.55*Xcm + 1.45*Xbm + 1.25*Xpm + 1.70*Xam
```

Decision-making examples using linear programming (continued)

• Constraints –

Consists of quantity constraints for each ingredient and constraints on the ratio of sausage ingredients.

```
subject to
Xcr + Xcb + Xcm ≤ 200 (닭고기의 수량 조건)
Xbr + Xbb + Xbm ≤ 300 (소고기의 수량 조건)
Xpr + Xpb + Xpm ≤ 150 (돼지고기의 수량 조건)
Xar + Xab + Xam ≤ 400 (전분의 수량 조건)
0.9*Xbr + 0.9*Xpr - 0.1*Xcr - 0.1*Xar ≤ 0 (일반 소시지의 비율 조건)
0.8*Xcr -0.2*Xbr - 0.2*Xpr - 0.2*Xar ≥ 0 (일반 소시지의 비율 조건)
0.25*Xbb - 0.75*Xcb - 0.75*Xpb - 0.75*Xab ≥ 0 (소고기 소시지의 비율 조건)
Xam = 0 (고기만 소시지의 비율 조건)
0.5*Xbm + 0.5*Xpm - 0.5*Xcm - 0.5*Xam ≤ 0 (고기만 소시지의 비율 조건)
Xij ≥ 0
```

Decision-making examples using linear programming (continued)

• Overall linear programming model

```
Maximize 0.7*Xcr + 0.6*Xbr + 0.4*Xpr + 0.85*Xar
       + 1.05*Xcb + 0.95*Xbb + 0.75*Xpb + 1.20*Xab
       + 1.55*Xcm + 1.45*Xbm + 1.25*Xpm + 1.70*Xam

subject to
  Xcr + Xcb + Xcm ≤ 200 (닭고기의 수량 조건)
  Xbr + Xbb + Xbm ≤ 300 (소고기의 수량 조건)
  Xpr + Xpb + Xpm ≤ 150 (돼지고기의 수량 조건)
  Xar + Xab + Xam ≤ 400 (전분의 수량 조건)
  0.9*Xbr + 0.9*Xpr - 0.1*Xcr - 0.1*Xar ≤ 0 (일반 소시지의 비율 조건)
  0.8*Xcr -0.2*Xbr - 0.2*Xpr - 0.2*Xar ≥ 0 (일반 소시지의 비율 조건)
  0.25*Xbb - 0.75*Xcb - 0.75*Xpb - 0.75*Xab ≥ 0   (소고기 소시지의 비율 조건)
  Xam = 0 (고기만 소시지의 비율 조건)
  0.5*Xbm + 0.5*Xpm - 0.5*Xcm - 0.5*Xam ≤ 0 (고기만 소시지의 비율 조건)
  Xij ≥ 0

(Xij = 각 원료가 소시지의 생산에 투입되는 수량. 여기서
   i = c, b, p, s (c: chicken, b: beef, p: pork, s: starch),
   j = r, b, m (r: regular, b: beef, m: meat) )
```

Deriving the Optimal Solution for Linear Programming with PuLP

- Import Pulp, define the problem name as 'sausage_problem', and the goal is Set to maximize

```
1  from pulp import *
2  LP = LpProblem("sausage_problem", LpMaximize)
```

- Definition of Decision

Variable – Prevent negative production by setting the lower bound of the decision variable to 0.

```
1  Xcr = LpVariable("Xcr",0)
2  Xbr = LpVariable("Xbr",0)
3  Xpr = LpVariable("Xpr",0)
4  Xar = LpVariable("Xar",0)
5  Xcb = LpVariable("Xcb",0)
6  Xbb = LpVariable("Xbb",0)
7  Xpb = LpVariable("Xpb",0)
8  Xab = LpVariable("Xab",0)
9  Xcm = LpVariable("Xcm",0)
10 Xbm = LpVariable("Xbm",0)
11 Xpm = LpVariable("Xpm",0)
12 Xam = LpVariable("Xam",0)
```

Deriving the Optimal Solution for Linear Programming with PuLP (Continued)

- Input Objective Function –

  When using a single command across multiple lines in Python, add the '\' symbol at the end of the line.

```
1  LP += 0.7*Xcr + 0.6*Xbr + 0.4*Xpr + 0.85*Xar \
2      + 1.05*Xcb + 0.95*Xbb + 0.75*Xpb + 1.20*Xab \
3      + 1.55*Xcm + 1.45*Xbm + 1.25*Xpm + 1.70*Xam
```

- Added constraints – also

  expresses equality and inequality signs. The equality sign is represented as '==' in Python.

```
LP += Xcr + Xcb + Xcm <= 200
LP += Xbr + Xbb + Xbm <= 300
LP += Xpr + Xpb + Xpm <= 150
LP += Xar + Xab + Xam <= 400
LP += 0.9*Xbr + 0.9*Xpr - 0.1*Xcr - 0.1*Xar <= 0
LP += 0.8*Xcr - 0.2*Xbr - 0.2*Xpr - 0.2*Xar >= 0
LP += 0.25*Xbb - 0.75*Xcb - 0.75*Xpb - 0.75*Xab >= 0
LP += Xam == 0
LP += 0.5*Xbm + 0.5*Xpm - 0.5*Xcm - 0.5*Xam <= 0
```

Deriving the Optimal Solution for Linear Programming with PuLP (Continued)

- Model optimization and variable value output (Full code: ex16-2.py)

```
1  LP.solve()
2
3  for v in LP.variables():
4      print(v.name, '=', v.varValue)
5  print("Total profit is ", value(LP.objective))
```

```
Xab = 100.0
Xam = 0.0
Xar = 300.0
Xbb = 300.0
Xbm = 0.0
Xbr = 0.0
Xcb = 0.0
Xcm = 125.0
Xcr = 75.0
Xpb = 0.0
Xpm = 125.0
Xpr = 0.0
Total profit is  1062.5
```

Newsbender model

Overview of the Newsvendor Model

- Translated into newspaper vendor model, single-period inventory model, etc.

- Very simple, but the supply chain's production and sales planning (S&OP: Sales and Operations
  An important model that forms the foundation of large-scale decision-making systems, such as planning.

- Basic assumptions of the Newsbender model
  - Demand occurs only for a specific period. – The
    quantity demanded during the demand period is probabilistic.
  - Production must be completed before the demand period, and there is no additional replenishment during the
    demand period. – During the demand period, products are sold at a price higher than their
    production cost. – After the demand period ends, any remaining inventory is disposed of in its entirety at a price lower than its production c

Newsbender Model Overview (Continued)

- Asymmetry between understock and overstock situations
  - Understock cost: Since a shortage of inventory leads to the loss of sales opportunities, an opportunity cost equal to the difference between the product's selling price and production costs arises for every additional demand.
  - Overstock cost: Inventory remaining until the end of the demand period is a production cost

    Since it must be disposed of at the end of the period, a loss is incurred equal to the difference between the production cost and the residual value per unit of remaining inventory.

- Normal selling price per unit – , production costs , If we let the residual value be,

  Shortage cost: = ÿ – Excess cost: = ÿ

- Examples of the Newsbender Model
  - Assuming you buy newspapers at the distribution center at dawn for 100 won each (= 100)
  - You can sell newspapers for 500 won for today (= 500), but the remaining until the end of today Newspapers must just be thrown away (= 0)

$$C_u = p - c = 500 - 100 = 400$$
$$C_o = c - s = 100 - 0 = 100$$

  - Demand is predicted to average 300 copies: it could be more or less than 300 copies

Newsbender Model Overview (Continued)

• News Vendor Decision-Making Structure

– Demand may be more or less than 300 copies – If another customer arrives when all the newspapers are sold out, restocking is impossible, resulting in a profit of 400 won.

Losing ÿ If insufficient, a 400 won loss

– Conversely, since unsold newspapers by the end of the day must simply be thrown away, a loss of 100 won per copy is incurred .

If there is a balance, a 100 won loss

– Since running out results in a greater loss, it is reasonable to buy just enough newspapers to leave a slight surplus ÿ It is recommended to order more than 300.

• For reference, simulations conducted on real people show a tendency to align with demand forecasts rather than the rational optimal; that is, they tend to order an amount equal to the predicted average demand.

• Assume that demand follows a normal distribution with a mean of 300 and a standard deviation of 150.

– To determine exactly how many copies to order, it is necessary to predict the probability distribution of demand – If the probability density function of demand is ( ), the expected profit ÿ when the order quantity (production quantity) is ( )

$$\Pi(Q) = -cQ + \int_0^Q \{px + s(Q-x)\}f(x)dx + \int_Q^\infty pQf(x)dx$$

– Although the exact profit can be calculated by expanding this formula, calculating the profit through simulation using Python

Decided to go for a hike

Newsbender model simulation

• Related costs and , setting

```
1  import numpy as np
2  p=500
3  c=100
4  s=0
5  Cu=p-c
6  Co=c-s
```

• Generation of demand

– When the mean is mu and the standard deviation is sigma, the normal segment having that mean and standard deviation The demand quantity x following the path can be generated by np.random.normal(mu, sigma).

```
1  mu=300
2  sigma=150
3  x = np.random.normal(mu, sigma)
4  if x<0: x=0
```

– (Line 4) If demand is negative, set it to 0

Newsbender Model Simulation (Continued)

- Profit calculation when demand is x

```
1  profit = -c*Q
2  if x<=Q: profit += p*x + s*(Q-x)
3  else:profit += p*Q
```

– (Row 1) Expend c*Q on initial purchase costs

– (Row 2) If demand x is less than or equal to the order quantity Q, generate revenue of p per unit for x units, and recover a residual value of s per unit for

remaining inventory Qx – (Row 3) If demand exceeds the order quantity, sell only up to the order quantity Q, generating revenue of p per unit. No residual quantity

Newsbender Model Simulation (Continued)

• Repeat the above situation 100,000 times and output the average profit –

if the order quantity Q=300, the expected profit is approximately

Calculated as 90,725 won • Since it

is calculated using a random number, there may be a slight difference

each time it is run

```
Order Quantity =  300
Expected profit =  90725.46723097205
```

– You can find the order quantity that maximizes profit by adjusting the order quantity.

예제 ex16-3.py

```
1  import numpy as np
2  p=500
3  c=100
4  s=0
5  Cu=p-c
6  Co=c-s
7  mu=300
8  sigma=150
9
10 Q=300
11 n=100000
12 sum = 0
13 for i in range(n):
14     x = np.random.normal(mu, sigma)
15     if x<0: x=0
16     profit = -c*Q
17     if x<=Q: profit += p*x + s*(Q-x)
18     else: profit += p*Q
19     sum += profit
20 avg = sum/n
21 print("Order Quantity = ", Q)
22 print("Expected profit = ", avg)
```

Optimal order quantity for the news vendor model

• Optimal service level (threshold) of the news vendor model

– Meaning that it is optimal to match a critical ratio of the total demand.

$$\alpha = \frac{C_u}{C_o + C_u}$$

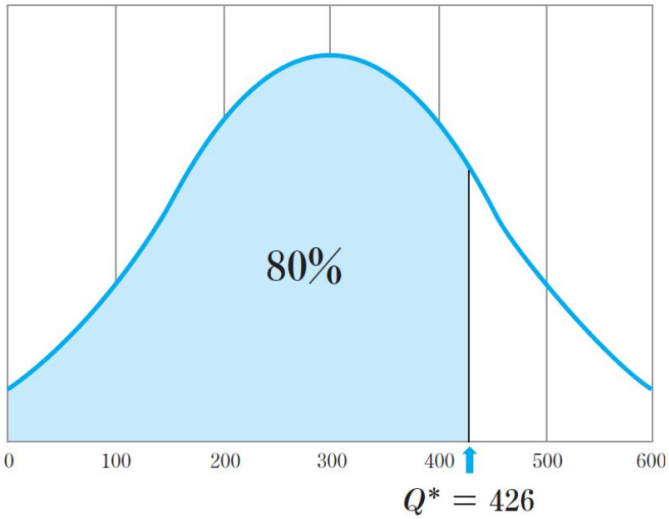
– Example) In the case of the newspaper boy mentioned earlier, determining an order quantity that satisfies 80% of the demand is optimal

$$\alpha = \frac{C_u}{C_o + C_u} = \frac{400}{100 + 400} = 0.8$$

• Determining the optimal order

quantity – Find the order quantity where the cumulative area under the probability density function curve equals the critical rate. • Using Excel's NORM.INV function – If the

demand follows a normal distribution with a mean of 300 and a standard deviation of 150, NORM.INV(0.8, 300, 150) **becomes** 426.



[그림 16–6] 최적 주문량의 결정

• If Q=426 is set in the simulator implemented earlier, the expected profit increases by more than 10% to approximately 99,650 won.

```
Order Quantity =  426
Expected profit =  99649.7226621867
```

– Confirming the importance of considering demand variability in decision-making

Data generation for artificial neural network application

• Demand does not occur purely probabilistically but is influenced by other external factors

Assuming the case

• Artificial neural networks can be utilized to forecast demand in situations involving complex influencing factors – Artificial neural networks generate approximation functions by finding hidden patterns between feature values and output values – However, since simply predicting demand values is insufficient, artificial neural networks [target] not demand but optimal

Need for a method to train the model to directly predict production volume •

Apply the model presented by Oroojlooyjadid et al. (2020)

• Naengmyeon Restaurant

Example – Assume that the demand for naengmyeon increases as the temperature rises, and that when the temperature is t, the average demand for naengmyeon at this restaurant is 5t + 50. – Additionally, the demand for naengmyeon is variable; normally, it fluctuates by approximately ±50% relative to the average demand, but this variability increases to ±60% when it rains. – If temperature t and weather w (w=0 for normal, 1 for rain), then the daily demand d is

$$\mu_d = 5t + 50$$

$$\sigma_d = \begin{cases} 0.5\mu_d, & if\ w = 0 \\ 0.6\mu_d, & if\ w = 1 \end{cases}$$

It appears as a normal distribution

Data Generation for Artificial Neural Network Application (Continued)

• Assuming the temperature is randomly generated with uniform probability between 0 and 30 degrees, and the weather has a 70% probability of being normal (w=0) and a 30% probability of rain (w=1), generate 10 requests for a cold noodle restaurant – (rows 3–4) store the temperature in temp and the weather in weather, respectively

Create and input n values of uniform and binomial distributions.

– (Line 5) Initialize the array `demand` to store demand with n elements

– (Line 7) Calculate

the average demand for each element – (Lines 8–9) Calculate the

standard deviation of demand based on whether the weather is 0 or 1

– (Lines 10–12) Generate

a random number from a normal distribution based on the mean and standard deviation and store it in `demand[i]` – The result

is generated and printed sequentially as follows: temperature, weather, and corresponding demand

amount • Since it is a random number, there may be differences

```
[ 2.1806825   0.75857485  8.32267316  9.05014197 16.89969309 23.35055747
 19.74189414 16.72264722  8.50543526  4.31712682]
[0 0 1 0 1 1 0 0 1 0]
[ 83.88534   27.00036  112.39334   48.647255  25.693317 103.13888
 176.96065  102.95215  115.89846   56.839222]
```

```
1  import numpy as np
2  n=10
3  temp = np.random.uniform(0, 30, n)
4  weather = np.random.binomial(1, 0.3, n)
5  demand = np.zeros(n, dtype='float32')
6  for i in range(n):
7      mu_d = 5*temp[i] + 50
8      if weather[i]==0: sigma_d = 0.5 * mu_d
9      else: sigma_d = 0.6 * mu_d
10     d = np.random.normal(mu_d, sigma_d)
11     if d<0: d=0
12     demand[i] = d
13 print(temp)
14 print(weather)
15 print(demand)
```

Simulation of production at average demand

• Business Setup – Assume

the cost of a bowl of cold noodles is 2,000 won and the selling price is 8,000 won. –

Assume leftover noodles are discarded, resulting in a salvage value of 0.

– Simulation of expected profit when producing at average demand ÿ Order quantity 125, expected profit approximately 545,344 won

Demand generation

Profit calculation when Q is set to average demand

예제 ex16-4.py

```
1  import numpy as np
2  n=100000
3  temp = np.random.uniform(0, 30, n)
4  weather = np.random.binomial(1, 0.3, n)
5  demand = np.zeros(n, dtype='float32')
6  for i in range(n):
7      mu_d = 5*temp[i] + 50
8      if weather[i]==0: sigma_d = 0.5 * mu_d
9      else: sigma_d = 0.6 * mu_d
10     d = np.random.normal(mu_d, sigma_d)
11     if d<0: d=0
12     demand[i] = d
13
14 p=8000
15 c=2000
16 s=0
17 Cu=p-c
18 Co=c-s
19
20 sum_profit = 0
21 sum_Q = 0
22 for i in range(n):
23     mu_d = 5*temp[i] + 50
24     Q = mu_d
25     d = demand[i]
26     profit = -c*Q
27     if d<=Q: profit += p*d + s*(Q-d)
28     else: profit += p*Q
29     sum_Q += Q
30     sum_profit += profit
31
32 avg_Q = sum_Q/n
33 avg_profit = sum_profit/n
34 print("Production Quantity = ", avg_Q)
35 print("Expected profit = ", avg_profit)
```

Implementation of Production Planning Optimization Using Artificial Neural Networks

- Application of Artificial Neural Networks to Production Planning

  Optimization – Considering Cost Asymmetry in Loss Function (Oroojlooyjadid et al., 2020)

- Limitations of general loss functions –

  Since production forecasting is a continuous value, mean square error is generally used as the loss function; however, this is symmetric, meaning the loss is the same whether the predicted value is larger or smaller than the correct value. – Therefore, if training is done to minimize this loss function value, the neural network will later [address] the learned correct value Demand is predicted by optimizing to predict the value closest possible – for example, if this neural network is trained on demand using temperature and weather, it will later predict the demand corresponding to the temperature and weather similarly.

- To train an artificial neural network to calculate the optimal production amount instead of predicting demand, hand

  The real function must change – if the production quantity determined by the neural network is greater than demand, the loss should be the cost of excess inventory multiplied by the amount of excess inventory; if the production quantity is less than demand, the loss should be the cost of shortage inventory multiplied by the amount of shortage inventory.

$$Loss(Q,d) = \begin{cases} C_o \cdot |Q-d|, & \text{if } Q \geqq d \\ C_u \cdot |Q-d|, & \text{if } Q < d \end{cases}$$

Production Planning Optimization Using Artificial Neural Networks (Continue

- Implementation of a custom loss function

```
1  p=8000
2  c=2000
3  s=0
4  Cu=p-c
5  Co=c-s
6  import tensorflow as tf
7  def inventory_loss(demand_true, q):
8      stock = q-demand_true
9      is_understock = stock < 0
10     inventory_error = tf.abs(stock)
11     understock_loss = Cu*inventory_error
12     overstock_loss = Co*inventory_error
13     return tf.where(is_understock, understock_loss, overstock_loss)
```

– (Line 8) Subtract realized demand from production and store the result in

stock. – (Line 9) is_understock takes a value of 1 when stock is negative (in a state of insufficient stock), and 0 otherwise. – (Line

10) Calculate the absolute value of the difference between production and realized demand and store it in

inventory_error. – (Line 13) tf.where returns the second argument if the first argument is 1, and the third argument otherwise. • Therefore,

the value of understock_loss is returned if there is an insufficient stock, and the value of overstock_loss is returned if there is an excess stock.

Production Planning Optimization Using Artificial Neural Networks (Continue

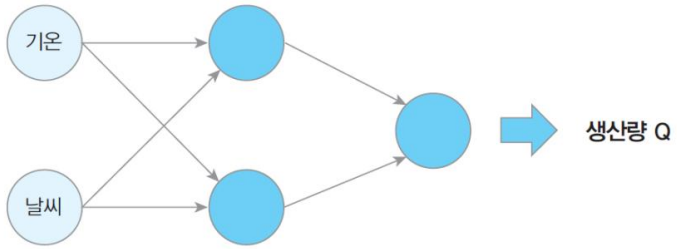
• Generate training data

```
1  x_train = np.zeros((n,2), dtype='float32')
2  x_train[:,0] = (temp - min(temp))/(max(temp)-min(temp))
3  x_train[:,1] = weather
4  demand_train = (demand - min(demand))/(max(demand)-min(demand))
```

– (Row 2) Scale temperature temp to between 0 and 1 and place it in the first column of x_train

– (Row 3) Place weather weather in the second column of

x_train – (Row 4) Place ground truth demand data into demand_train



[그림 16-7] 생산량을 도출하는 인공신경망 구조

• Neural Network

Configuration – 2 input data points (temperature,

weather) – The output layer outputs a continuous

value (production quantity) – The hidden layer is a single layer with 2 neurons

```
1  from tensorflow import keras
2  model = keras.Sequential([
3      keras.layers.Dense(2, input_shape=(2,), activation='sigmoid'),
4      keras.layers.Dense(1, activation=None),
5  ])
```

Production Planning Optimization Using Artificial Neural Networks (Continue

• Define the optimization algorithm and loss function, and perform training – at this stage,

set the loss function to `inventory_loss`, the custom loss function implemented earlier.

```
model.compile(optimizer='adam', loss=inventory_loss)
model.fit(x_train, demand_train, epochs=20)
```

• Output the average production volume calculated by constructing 10,000 test data records – The production volume

determined by the neural network differs significantly from the average demand of 125.

```
n_test=10000
temp_new = np.random.uniform(0, 30, n_test)
weather_new = np.random.binomial(1, 0.3, n_test)
x_test = np.zeros((n_test,2), dtype='float32')
x_test[:,0]=(temp_new - min(temp))/(max(temp)-min(temp))
x_test[:,1]=np.random.binomial(1, 0.3, n_test)
pred = model.predict(x_test)
Q_prop = pred*(max(demand)-min(demand))+min(demand)
print("Mean of Q_prop = ", np.mean(Q_prop))
```

```
Mean of Q_prop =  169.59798
```

Performance simulation of artificial neural network production planning

• Since the previously written code contains the test input data x_test and the corresponding neural network output Q_prop, simulate performance using these values – (Lines 4–6) Calculate the mean and standard deviation of demand based on temperature and weather.

– (Lines 7–8) Generate demand based on mean and standard deviation – (Lines 9–12) Corresponding demand d and neural network-calculated production quantity Profit calculation corresponding to Q_prop

```
1  sum_profit = 0
2  sum_Q = 0
3  for i in range(n_test):
4      mu_d = 5*temp_new[i] + 50
5      if weather_new[i]==0: sigma_d = 0.5 * mu_d
6      else: sigma_d = 0.6 * mu_d
7      d = np.random.normal(mu_d, sigma_d)
8      if d<0: d=0
9      Q = Q_prop[i]
10     profit = -c*Q
11     if d<=Q: profit += p*d + s*(Q-d)
12     else: profit += p*Q
13     sum_Q += Q
14     sum_profit += profit
15 avg_Q = sum_Q/n
16 avg_profit = sum_profit/n
17 print("Production Quantity = ", avg_Q)
18 print("Expected profit = ", avg_profit)
```

• Execution result

```
Production Quantity =  [170.34726]
Expected profit =  [587996.61056]
```

– Average production volume 170, expected profit 587,997 KRW – When production volume was set equal to the average demand, the expected profit was approximately 545,344 KRW • Profit improvement rate approximately 7.8%